TECHNICAL ASSIGNMENT · HUNAR.AI

Hunar Recruiting

Two recruiting applications sharing one voice pipeline: AI phone screening of applicants, and consent-gated outreach to people who never applied.

Live demo	https://hunar-aryan-s-projectsss1.vercel.app
Password	`ledger-copper-tundra-20`
API schema	https://hunar-rvf8.onrender.com/docs
Repository	https://github.com/Rn-sripathi/Hunar
Author	Rishwanth Aryan Sripathi

Backend FastAPI on Python 3.12 · frontend Next.js and TypeScript with shadcn/ui · Postgres on Neon · 498 tests, ruff and mypy strict clean

Overview

Two recruiting applications that share one voice pipeline.

Screening — a recruiter defines a role and the questions they want asked, loads applicants, and launches calls. Hunar's voice agents conduct the interviews in Hindi, Tamil or English, and the extracted answers land in a dashboard with transparent scoring and a ranked shortlist.

Sourcing — paste a job description, find people who never applied, and reach the ones you are permitted to call. Same voice agents, same results table, a very different conversation.

They are one deployment because roughly seventy percent of the work is identical: the voice client, the webhook receiver, correlation tokens, answer coercion, the shared call table and the polling hook are reused whole. What differs is who is on the other end of the phone, and that difference is where all the interesting design lives.

Built for the Hunar.ai assignment. The written answer to the third question is in docs/attendance-without-apps.md.

The demo is password protected because its database holds real candidates' names and phone numbers. The API schema at `/docs` is deliberately open: it carries no data and reading it is a feature for anyone reviewing this.

Outbound calling is switched **off** on the deployed instance: `DEMO_ALLOWLIST` is empty, so the sourcing app will source people and refuse to phone any of them. That is the intended posture for a public URL, and the refusal is shown against each person with its reason rather than hidden.


Results dashboard

What works today

Screening applicants

- **Paste a job description and the whole form fills itself in**, including the screening questions. Falls back to keyword extraction when no model key is configured, and always says which of the two produced the fields.
- Create a role from a job description and a list of screening questions.
- **See the exact call script before anyone is phoned.** The generated agent prompt and extraction schema are shown beside the form as you type.
- Add candidates by hand or import a spreadsheet, with bad rows reported rather than silently dropped.
- Launch screening calls, with a confirmation stating how many real phone calls are about to be placed.
- Watch call status update live.
- Review answers in a table whose columns are built from that role's questions, with the coerced value and the candidate's literal words side by side.

- Transparent scoring, ranked shortlist, CSV export.

Sourcing and outreach

- **Paste a job description and get editable search filters.** Extraction is free and reversible; the search costs a provider credit. Keeping them apart is what lets a wrong interpretation be corrected before it is paid for.
- Search real people, ranked for outreach priority, with **the exact provider query one click away** so a surprising result set can be explained rather than argued about.
- **Every fit score explains itself**, component by component, with a standing note that it ranks who to approach first and must never gate a hiring decision.
- **People the app will not call are shown, with the reason**, never hidden. Demonstrating the gate firing is the point of having one.
- Bind a sourced person to a consented number, assemble a campaign, and read **the exact opening sentence a stranger will hear** before pressing call.
- Cold-call answers land in the same results table as screening answers, because the table builds itself from whatever the agent was asked to extract.

Sourcing is broad. Calling is narrow.

This is the central design decision of the second app, and it came from research that overturned the obvious approach.

No provider named in the brief will return a mobile number on a free tier.

Provider	Person search	Phone number	Status
People Data Labs	Yes — 100 credits/month, no card	**No** — contact fields return `true` / `false` on free tiers since v29.0	Implemented
Apollo.io	Plan-gated	Costs credits where available	API access depends on plan; their docs say upgrade
Proxycurl	—	—	**Shut down July 2025** after the LinkedIn suit
Coresignal	Yes, trial	**No** — public-web sourcing yields no personal phones	Not pursued

Checked 7 September 2026. Free tiers move; the shape of the conclusion has not.

So the chain the brief describes — search, then phone, then call — cannot be closed on free credits by anyone. Rather than fake it, the product splits the two permissions: **sourcing is real and unrestricted, calling is gated.**

That is also the right answer independent of the tier. Cold-calling someone whose number came from a broker engages India's TCCCPR rules, which require DLT sender registration this demo does not have; People Data Labs' own acceptable-use policy forbids using their data for employment decisions; and PDL

settled a $6.36M class action in 2025 over exactly this kind of phone data. The constraint and the ethics point the same way, which is usually a sign the design is right.

How the gate works. Outbound calls go only to numbers on an allowlist supplied through the environment, which the running app cannot extend. A sourced person becomes callable only when an operator binds them to one of those already-permitted numbers — which is how consent genuinely arrives, via a reply or a referral, and never from the fact that someone was findable.

It is enforced at three depths:

1. The service refuses, with a reason written for a person.
2. The UI shows the refusal next to the person it refers to.
3. `outreach_target.allowlist_id` is `NOT NULL`, so a bug that skips the service layer fails at insert rather than placing a call.

The check runs **twice** — once when a campaign is assembled, once immediately before each call — because a campaign built at 18:55 can still be draining at 19:05, and calling hours are not advisory. Suppression stores only a SHA-256 hash: honouring "never call me again" should not require retaining the number of the person who asked.

With no credentials at all, a fixture provider serves the search screen with real filtering, ranking and pagination over a built-in set, so the whole flow is demonstrable without signing up for anything.

Five things the API taught us

The published documentation left several things unstated. These were established by reading the live OpenAPI schema, and each one changed the design.

Finding	Consequence
`result_schema` is a flat map of field name to type hint, and it lives on the **agent**, not the call	One agent must be created per job role. A shared agent could not carry per-role extraction at all
Extracted values come back as **strings** whatever type was declared. A `"boolean"` field returns `"Yes"`	A coercion layer is mandatory, not a nicety
The status webhook fires **only at terminal states**	Live progress cannot come from webhooks. Polling is infrastructure here, not a fallback
The `call_result_done` payload contains **no call identifier**	Correlation moved into the callback URL as an opaque token
Prompt templating uses **single braces**, and `custom_data` is strings only	Every pasted job description must be brace-stripped, or a salary range corrupts what the agent says aloud

Also: agent creation accepts no `conclusion` or `silence_response` despite the prose mentioning them, callbacks must be HTTPS, guardrails need three distinct days and a three-hour window, and `retry_interval_hours` accepts only 0, 3, 6, 9, 12 or 24.

Architecture

```
apps/api          FastAPI. Two independent domains sharing core + the SDK.
  app/core        Config, logging, errors, shared call and webhook tables
  app/hiring      Screening applicants
  app/people      Sourcing and outreach
  app/webhooks    Signed webhook receiver, shared by both domains
apps/web          Next.js App Router, TypeScript strict, shadcn/ui
packages/hunar-sdk  Typed voice client, webhook verification, demo client
```

`hiring` and `people` may never import each other. That is enforced by an architecture test rather than a comment, because the rule is directional and a lint rule cannot express it.

Frontend types are **generated from the backend's OpenAPI schema** (`npm run gen:api`), so a renamed field is a compile error rather than an `undefined` at runtime.

Decisions worth defending

A call POST is never retried. If it times out the call may still have been placed, and `GET /calls/` cannot be filtered by `request_id` to check. Retrying would risk a second real phone call to a real person, so the attempt is parked as `UNKNOWN` and the reconciler resolves it.

Reconciliation runs lazily on read, not in a worker. No scheduler, it survives a host that sleeps between requests, and it cannot drift out of step with what the user is looking at.

Editing a role's questions after calls exist is refused. Stored answers only mean something against the schema that produced them. Silently changing the questions would silently change what past results mean.

Unparseable is never "no". An answer the coercion layer cannot read becomes null, not false. Reading "I'm not sure" as a refusal would reject a candidate for hesitating, which is the one error here with a human cost.

An unanswered hard requirement never disqualifies anyone. A knockout fires only on a clear, judged failure, and always shows its reason.

Scoring is rule-based, not a model judgement. This ranks people for employment. A weighted rule can be audited, reproduced and challenged, and it explains itself field by field. Every score in the UI comes with its reasoning.

Demo mode is honest. When the voice key expires the app serves simulated calls and says so in a banner on every screen. The demo client returns strings and fires webhooks only at terminal states exactly as the real API does, so it exercises the production code path rather than bypassing it.

Running it locally

Requires uv, Node 20+, and a Postgres database (a free Neon project works).

```
cp .env.example .env          # then fill in DATABASE_URL
uv sync --all-packages
cd apps/api && uv run alembic upgrade head && cd ../..

# terminal 1
uv run uvicorn app.main:app --reload --port 8000

# terminal 2
cd apps/web && npm install && npm run dev
```

Open http://localhost:3000. With `HUNAR_MODE=mock` and the default `PEOPLE_PROVIDER=fixture`, both applications run end to end with no credentials at all. The two switches are deliberately independent, so an expired voice key does not force fake people data or the reverse.

To source live people, set `PEOPLE_PROVIDER=pdl` and `PDL_API_KEY`. To call anyone, put a number you control in `DEMO_ALLOWLIST` — nothing is dialable until you do, and that is the point.

To use the real voice API, put the key in `HUNAR_API_KEY`, set `HUNAR_MODE=auto`, and expose the backend over HTTPS so webhooks can reach it:

```
ngrok http 8000        # then set PUBLIC_API_BASE_URL to the https URL
uv run python scripts/spike.py                       # read-only probes
uv run python scripts/spike.py --call +91XXXXXXXXXX --confirm   # one real call
```

The spike answers the blocking questions before any product code depends on them, and saves every response as a fixture.

Tests

```
uv run pytest                # 498 tests
uv run ruff check . && uv run mypy packages/hunar-sdk/src apps/api/src scripts
cd apps/web && npm run typecheck && npm run lint && npm run build
```

Depth over breadth. The heavily covered parts are the ones where being wrong costs something: webhook signature verification, answer coercion, the scoring engine, and prompt generation. The suite runs on SQLite so it needs no database server; the models declare JSON columns as a variant that is JSONB on Postgres.

Deploying

Backend to Render, frontend to Vercel, database on Neon.

1. Push to GitHub. Render reads `render.yaml`.
2. Create the Render service, and set the secrets marked `sync: false`.
3. Set `PUBLIC_API_BASE_URL` to the service's own URL and redeploy. It is the root of every webhook callback URL, and Hunar rejects non-HTTPS callbacks.
4. Deploy `apps/web` on Vercel with `NEXT_PUBLIC_API_BASE_URL` pointing at it.
5. Add the Vercel domain to `CORS_ORIGINS` on Render, with no trailing slash. This is not optional and it is not obvious when wrong: an empty value permits *no* origin, so both services report perfect health while every request fails inside the browser. Comma-separate several if you also want preview deployments to work, since each Vercel deployment URL is a separate origin.

Deploy the backend first: step 3 depends on knowing its URL.

Security

The Hunar key is also the HMAC secret for inbound webhooks, so a browser-visible copy would let anyone forge webhooks that mutate call records. It therefore lives only in the backend process. `.gitignore` was written before `git init`, so no credential has ever entered this repository's history. Logs redact keys, signatures and phone numbers by processor rather than at each call site.

A full phone number never leaves the server. Only the masked form is in any response. That is worth stating precisely, because the first version sent both and let the browser choose which to render — masking is decorative while the unmasked value travels beside it, and a single misconfiguration then leaks every number. The field is absent from the API rather than merely unused by the UI.

The deployment is behind a shared password, held as a signed HttpOnly cookie and compared in constant time. Health checks stay open, or the platform declares the service dead. The API schema stays open deliberately: reading it is a feature for a reviewer and it carries no data. Webhooks stay open because Hunar cannot log in, and that path is authenticated by an HMAC over the raw request body, which is stronger than a password.

This is a gate, not authentication. There are no accounts and no record of who did what — see the limitations below.

Two ordering details are load-bearing rather than stylistic, and both are pinned by tests because neither fails visibly on the server:

- CORS is the **outermost** middleware. Anything able to reject a request has to sit inside it, or the rejection carries no `Access-Control-Allow-Origin` header and the browser discards a response it was not permitted to read. A `401` telling the frontend to ask for a password then becomes an unexplained network failure.
- The webhook route reads the raw body **before** verifying, and no middleware may rewrite it, because the signature covers the exact bytes received.

Put the database near its users

This is the single biggest thing affecting how the app feels, and it is a hosting choice rather than a code one.

Measured from India against a Neon project in `us-east-2`:

Nothing in the application can make up for that. Create the Neon project in the region nearest your users, `ap-south-1` for India, and the same operations drop to tens of milliseconds. The UI hides some of it with optimistic updates, but hiding latency is not the same as not having it.

Limitations, honestly

- **Reconciliation is in-process**, so the API is pinned to one instance. Real scale needs a separate worker holding a lock. The seam is there; the worker is not.
- **The screening agent has been heard; the outreach agent has not.** One real screening call was placed end to end — 67 seconds, answers extracted, score and recording returned. The cold-outreach script has been read aloud only in preview. Prompt quality is genuinely knowable only from real calls, and that one is still owed.
- **No authentication.** The shared password stops a public URL being readable; it does not identify anyone. Real use needs per-recruiter accounts and an audit trail of who rejected whom, because a screening decision is about a person and should be attributable.
- **Recordings are proxied, not stored.** Provider URLs are likely to expire with the key.
- Retry policy, calling-hours guardrails and per-candidate language are supported by the API client but not yet exposed in the UI.
- **Deferred targets are not dialled automatically.** A campaign assembled outside calling hours marks its targets deferred and waits for someone to press launch again. Draining them when the window opens needs the same background worker the reconciler wants, and for the same reason it is not there yet.
- **The people-search provider returns no phone numbers**, by design of its free tier. See the sourcing section above; this shapes the product rather than limiting it.

Repository

```
docs/build-plan.md               the plan this was built against
docs/attendance-without-apps.md  the third question, answered
scripts/spike.py                 day-zero API probe, run before product code
scripts/dump_openapi.py          regenerates the frontend types from the backend
scripts/build_docs_pdf.py        renders this README to a print-quality PDF
```

Appendix: Attendance without smartphones

The problem. A thousand people work across a hundred sites, roughly ten per site. None of them has a smartphone, so there is no app, no GPS, no QR code, no selfie check-in. Large language models are available and cheap. What do you build?

The short answer: **the missed call is the primitive, and the language model is the exception handler.** Almost everything below follows from refusing to put the model in the hot path.

---

1. What these people actually have

"No smartphone" is not "no phone". In Indian frontline work the near universal device is a feature phone, and it can do four things:

Capability	Cost to the worker	Carries identity?
Missed call	**Free**	Yes — the caller ID
SMS	~₹0.10	Yes
USSD (`*123#`)	Free on most plans	Yes
Voice call	Paid, unless we call them	Yes

The missed call is the interesting one, and it is worth being precise about why. It is free for the worker, it works on every handset ever sold, it needs no literacy, no data connection, no menu, no training beyond "ring this number and hang up", and the caller ID is an identifier the worker cannot easily forge and did not have to remember.

India already runs on this. Missed-call campaigns are how banks deliver balances and how political parties count supporters. Building attendance on it is not a clever hack; it is using the channel these users already understand.

So the base design is one phone number per site, and the worker gives it a missed call at the start and end of their shift. We capture three things for free: **who** (caller ID), **where** (which number was dialled), **when** (the timestamp).

One number per site for a hundred sites is a trivial cost. The check-ins themselves cost nothing at all, because the call is never answered.

---

2. Where a language model actually earns its place

The tempting design is to have a voice agent ring all thousand people every morning. That is worse on every axis: it costs real money per call, it annoys people at 6 a.m., it is trivially gamed by saying "yes" to a robot, and it puts a probabilistic system in the path of something that should be deterministic.

A missed call is already a perfect, unambiguous signal. Do not send an LLM to interpret it.

The model belongs where there is genuine ambiguity, which on a normal day is about five to ten percent of the roster:

Nobody checked in. Thirty minutes past shift start, the system rings the worker. Not to nag — to find out what happened. "Are you on your way, or is something wrong?" The answer is the valuable part, and it is the part conventional attendance systems throw away. Most of them record a bit: present or absent. What a site manager actually needs is *"bus broke down at Silk Board, forty minutes out"* versus *"child is ill, not coming"* versus *"I have been here since six, my phone is dead"*. Those three absences require three completely different responses, and only the third one is not an absence at all.

This is exactly the extraction problem the rest of this repository solves: unstructured speech in, structured fields out.

```
"bus kharab ho gaya, Silk Board pe, chalis minute lagega"
  → { status: DELAYED, eta_minutes: 40, reason: TRANSPORT,
      raw: "bus kharab ho gaya, Silk Board pe, chalis minute lagega" }
```

Language and literacy. A voice agent that speaks Hindi, Tamil, Kannada and the mixture people actually use is more accessible than any SMS menu, and far more accessible than a USSD tree. Reading is a real constraint in this workforce; speaking is not.

Reconciliation with supervisors. At a hundred sites, the supervisor's headcount and the check-in log will disagree daily. Resolving that normally needs a literate person at a computer. Instead the agent calls the supervisor, reads out the three names in dispute, and accepts a spoken correction. A back-office task becomes a two-minute phone call.

Everything else stays dumb on purpose. Detecting that one handset checked in for six different workers is a `GROUP BY`, not a language model. Use the cheap deterministic thing wherever a cheap deterministic thing works.

---

3. Proving presence, not just identity

A missed call proves who is calling. It does not prove where they are. Someone can ring the Whitefield number from their bed. This is the real problem, and it deserves an honest answer rather than a clever one.

Perfect verification without hardware is impossible. The goal is not to make cheating impossible; it is to make it effortful enough not to be worth it, and to surface the residue to a human. Three layers, in increasing cost:

1. **A rotating site code.** A short code, changed daily, written on the board at muster or announced by the supervisor. The worker sends it by SMS instead of a missed call, or reads it to the agent. Knowing it implies having been at the site this morning. It is not unforgeable — someone can text it to a friend — but it converts casual absence into deliberate collusion, which is a much higher bar.

2. **Random voice audits.** A small random sample each day, perhaps two percent, gets a short call asking them to put the supervisor on, or to confirm something about the day's work. Unpredictability does most of the work here; the cost is negligible because the sample is tiny.
3. **Anomaly detection, no model required.** One handset checking in for several workers. Check-ins clustered within the same few seconds. A worker whose arrival precedes shift start by exactly the same interval every single day. These are queries, they are nearly free, and they point a human at the right site.

Where a site happens to have a landline, caller ID from a fixed line is strong location evidence and should be preferred. Cell-tower location from the telco would be stronger still, and I would not use it: it needs a carrier partnership, it costs real money, and it tracks people well beyond the workplace. That is a large privacy cost for a marginal improvement over a rotating code.

---

4. The asymmetry that should shape the whole design

Attendance determines pay. That makes the two failure modes profoundly unequal:

- A **false present** costs the employer a few hundred rupees.
- A **false absent** costs a worker a day's wages, and their trust in the system, permanently.

So the system must never automatically mark anyone absent. Unresolved means *flagged for a human*, never *not paid*. Every automated decision needs the raw evidence attached to it — the recording, the timestamp, the literal words — so that a worker who disputes it can be shown what happened rather than told what the system decided.

For the same reason the worker needs a way in. A missed call to a different number, and the agent reads back their own attendance for the week. Someone whose pay depends on a record should not have to ask permission to see it.

The provenance of each record should be stored alongside it, because not all attendance is equally well evidenced:

How it was recorded	Confidence
Missed call plus valid site code	High
Missed call only	Medium
Supervisor attested on the worker's behalf	Low, and auditable
Agent call, worker confirmed	Medium

A record that carries its own provenance can be argued with. A bare boolean cannot.

---

5. What it costs

Monthly, for a thousand workers across a hundred sites:

Item	Estimate
100 site numbers (DIDs)	~₹10,000
Missed calls, ~44,000/month	**₹0** — never answered
Agent calls at ~7% of check-ins, ~1.5 min each	~₹5,000
LLM inference on ~3,000 short calls	~₹3,000
Total	**~₹18,000/month, or ₹18 per worker**

Against biometric terminals at ₹5,000–15,000 per site, that is ₹5–15 lakh of capital expenditure before anyone has clocked in once, plus maintenance, power and connectivity at a hundred sites, plus the fingerprint-reader failure rate on hands that do manual work all day. The voice approach is roughly two orders of magnitude cheaper and works during a power cut.

---

6. What I would build first

Week one, and it needs no AI at all: one number per site, missed-call capture, a table of check-ins, and an SMS to each supervisor at shift start plus thirty minutes listing who is missing. This alone solves most of the problem, and shipping it first proves the channel works before any model is involved.

Week two: the outbound agent for missing check-ins, with structured extraction of the reason. This is where the system starts producing something a spreadsheet never could.

Week three: rotating site codes, the anomaly queries, the worker-facing "read me my attendance" line, and supervisor reconciliation by voice.

Sequencing matters here beyond ordinary iteration. If the missed-call layer is not solid, an LLM layered on top will mask its failures with plausible-sounding calls, and nobody will notice for a month.

---

7. What this does not solve

Being honest about the edges:

- **No phone at all.** A real minority. They need supervisor attestation, recorded as such, with the lower confidence that implies.
- **Shared handsets.** Common in households, and more common for women workers. One number cannot distinguish two people. Site codes help; a genuinely shared phone needs a second factor or attestation.

- **Determined collusion.** A supervisor who wants to mark a whole site present can. No attendance system without hardware survives a supervisor who is in on it; what this design can do is make the pattern visible in the data.
- **Network dead zones.** Some sites have no coverage. Those need the supervisor to batch-report from wherever signal exists, and the record should say so.
- **Number churn.** People change SIMs. The enrolment flow needs to be as easy as the daily flow, or the roster rots.

None of these is a reason not to build it. A system that covers ninety percent of a thousand people for ₹18 a head, and is honest about the other ten percent, is worth far more than one that claims to cover everybody and quietly gets some of them docked a day's pay.